

System do Zarządzania Rezerwacjami dla Małych Firm

Dokumentacja techniczna i użytkownika

Projekt Inżynierski

Projekt Zespołowy

Prowadzący: mgr inż. Aleksander Wojtowicz

Autor: Piotr Pająk, Karol Ogonowski, Dawid Sudoł, Milena Pasterczyk

Nr indeksu: pp131483, ko131480, ds131517, mp131485

Grupa: Tak

React

.NET

RabbitMQ

PostgreSQL

Docker

Rzeszów, 9 stycznia 2026

Uniwersytet Rzeszowski

Oświadczenie

My, niżej podpisani:

Piotr Pająk student Instytutu Informatyki

Karol Ogonowski student Instytutu Informatyki

Dawid Sudoł student Instytutu Informatyki

Milena Pasterczyk studentka Instytutu Informatyki

studenci Instytutu Informatyki Uniwersytetu Rzeszowskiego, oświadczamy, że niniejsza praca została wykonana przez nas samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Oświadczamy również, że przedstawiona praca nie była wcześniej przedmiotem procedur związanych z uzyskaniem zaliczenia na tej lub innej uczelni.

Rzeszów, 9 stycznia 2026

(podpis autora)

Streszczenie

Niniejsza praca opisuje proces projektowania i implementacji nowoczesnego systemu webowego do kompleksowego zarządzania rezerwacjami oraz strukturą biznesową przedsiębiorstwa. Aplikacja umożliwia użytkownikom dynamiczne planowanie wizyt, weryfikację dostępności terminów w czasie rzeczywistym oraz personalizację powiadomień, dostarczając jednocześnie zaawansowane narzędzia administracyjne do zarządzania kadrą i harmonogramami pracy.

System został zrealizowany w architekturze mikroserwisowej z wykorzystaniem platformy .NET 8 na backendzie oraz biblioteki React 19 na frontendzie. Warstwa danych opiera się na relacyjnej bazie PostgreSQL 15, a komunikacja asynchroniczna między usługami realizowana jest poprzez brokera komunikatów RabbitMQ. W projekcie wykorzystano technologię SignalR Hub do powiadomień real-time oraz standard JWT dla zapewnienia bezstanowego bezpieczeństwa i autoryzacji opartej na rolach (RBAC).

Całość rozwiązania została w pełni skonteneryzowana przy użyciu narzędzi Docker i Docker Compose, co gwarantuje izolację usług i powtarzalność środowiska. Dokumentacja szczegółowo omawia architekturę systemu, mechanizmy normalizacji danych poprawiające komfort użytkownika (UX) oraz proces automatyzacji wdrożeń realizowany poprzez pipeline CI/CD w środowisku GitHub Actions.

Słowa kluczowe: system rezerwacji, mikroserwisy, React 19, .NET 8, PostgreSQL, RabbitMQ, SignalR, Docker, JWT, Ocelot.

Spis treści

Streszczenie	2
Spis rysunków	5
Spis tabel	6
Spis listingów	7
1 Wprowadzenie	8
1.1 Kontekst i motywacja	8
1.2 Cel projektu	8
1.3 Zakres projektu	8
1.4 Użyte technologie	9
2 Architektura systemu	10
2.1 Architektura wysokopoziomowa	10
2.2 Wzorce architektoniczne i projektowe	11
2.2.1 API Gateway (Ocelot)	11
2.2.2 CQRS (Command Query Responsibility Segregation)	12
2.2.3 Architektura sterowana zdarzeniami (Event-Driven Architecture) . .	12
2.2.4 Wzorzec Repozytorium (Repository Pattern)	12
2.2.5 Logiczny model powiązań danych	12
3 Struktura bazy danych	14
3.1 Baza danych tożsamości (IdentityService DB)	14
3.2 Baza danych rezerwacji (ReservationService DB)	15
3.3 Baza danych powiadomień (NotificationService DB)	15
3.4 Integralność danych w architekturze rozproszonej	16
4 Implementacja kluczowych funkcjonalności	18
4.1 Bezpieczeństwo i zarządzanie tożsamością (JWT)	18
4.2 Zarządzanie strukturą biznesową i ofertą usług	19
4.3 Algorytm wyliczania dostępności i rezerwacji terminów	20
4.4 Asynchroniczny obieg zdarzeń i powiadomień	21
4.5 Dziennik audytowy i śledzenie zmian (<i>Audit Log</i>)	22
5 Warstwa prezentacji – Implementacja funkcjonalności Frontend (React)	24
5.1 Moduł Użytkownika – Proces rejestracji i rezerwacji	24
5.2 Zarządzanie profilem i Dashboard klienta	25
5.3 Moduł Firmowy – Zarządzanie uprawnieniami i kadrą	25

5.4	Zarządzanie harmonogramami i przerwami pracowników	26
5.5	Moduł Administracyjny i szablony powiadomień	27
5.6	Integracja z geokodowaniem i mapami	27
6	Konteneryzacja i środowisko uruchomieniowe	29
6.1	Konfiguracja wielokontenerowa (Docker Compose)	29
6.2	Charakterystyka zdefiniowanych usług	29
6.3	Zarządzanie zależnościami i Healthchecks	30
6.4	Izolacja sieciowa	30
6.5	Automatyzacja zarządzania systemem (Skrypty Manage)	31
7	Testowanie i proces CI/CD	32
7.1	Testy automatyczne	32
7.1.1	Testy warstwy backendowej (xUnit)	32
7.1.2	Testy warstwy frontendowej (Vitest)	32
7.2	Potok CI/CD w środowisku GitHub Actions	32
7.3	Weryfikacja integracyjna (Smoke Tests)	33
8	Podsumowanie i perspektywy rozwoju	34
8.1	Wnioski z realizacji projektu	34
8.2	Kierunki dalszego rozwoju systemu	34
8.3	Podsumowanie końcowe	35

Spis rysunków

1	Architektura mikroservisowa systemu rezerwacji	10
2	Architektura komunikacji międzyusługowej (Inter-Service Communication)	12
3	Logiczny model relacji między encjami mikroservisów Identity, Reservation i Notification.	13
4	Diagram związków encji (ERD) bazy danych mikroservisów Identity-Service	14
5	Diagram związków encji (ERD) bazy danych mikroservisów Notification-Service	16
6	Diagram związków encji (ERD) bazy danych mikroservisów Reservation-Service	17

Spis tabel

1	Stack technologiczny projektu	9
---	---	---

Listings

1	Implementacja punktów końcowych do zarządzania sesją (RefreshTokenController.cs)	18
2	Implementacja logiki wykrywania kolizji i walidacji slotów (AppointmentController.cs)	20
3	Mechanizm personalizacji powiadomień i obsługi szablonów (NotificationController.cs)	21
4	Implementacja bezpiecznego dostępu do logów audytowych (CompanyAuditController.cs)	23
5	Walidacja danych kontaktowych oraz filtrowanie slotów (BookingPage.js) .	24
6	Normalizacja kodu pocztowego i formatowanie numeru telefonu (AccountPage.js)	25
7	Logika weryfikacji uprawnień wewnątrz organizacji (CompanyPanel.js) . .	25
8	Obsługa komponentu wyboru dni tygodnia (CompanyPanel.js)	26
9	Mechanizm wstawiania tokenów do edytora szablonów (AdminPanel.js) . .	27
10	Geokodowanie adresu i generowanie widoku mapy (ServiceDetailsPage.js) .	27
11	Uruchomienie całego stosu technologicznego systemu	29
12	Fragment konfiguracji Docker Compose obsługujący zależności	30
13	Uruchomienie kompletnego środowiska za pomocą skryptu zarządczego . .	31

1 Wprowadzenie

1.1. Kontekst i motywacja

Dzisiaj tradycyjne metody, jak papierowe kalendarze czy proste tabele, stają się niewystarczające. Bardzo łatwo o błąd: wystarczy jedna pomyłka w zapisanej godzinie lub dwa terminy nałożone na siebie, by stracić klienta i pieniądze.

Współczesny biznes potrzebuje narzędzi, które automatyzują obsługę klienta i pozwalają zarządzać grafikami w czasie rzeczywistym. Ten system został zaprojektowany, aby pomóc właścicielom firm w codziennych obowiązkach. Umożliwia on łatwe śledzenie wolnych terminów i pozwala klientom samodzielnie rezerwować wizyty. Dzięki temu przedsiębiorca unika chaosu w kalendarzu, podnosi jakość obsługi i może skupić się na rozwijaniu swojego biznesu.

1.2. Cel projektu

Głównym celem projektu było stworzenie nowoczesnej, bezpiecznej i wydajnej aplikacji webowej klasy produkcyjnej, służącej do inteligentnego planowania wizyt oraz zarządzania zasobami firmy. System został zbudowany tak, aby z jednej strony maksymalnie uprościć klientowi drogę do rezerwacji terminu, a z drugiej – dostarczyć przedsiębiorcy zaawansowane narzędzie do pełnej kontroli nad grafikami i personelem.

1.3. Zakres projektu

Projekt obejmuje pełen cykl wytwarzania oprogramowania, od analizy wymagań po wdrożenie na serwerze produkcyjnym.

- **Backend:** C# .NET 8 [1], ASP.NET Core, Entity Framework Core
- **Frontend:** React 19 [2], Vite 7 [3], Axios 1.x [4]
- **Baza danych:** PostgreSQL 15 [5] (3 osobne bazy danych)
- **Messaging:** RabbitMQ 3.12 [6]
- **Komunikacja Real-time:** SignalR 8.0 [7]
- **Konteneryzacja:** Docker, Docker Compose [8]
- **Autentykacja:** JWT [9] (mechanizm Access + Refresh Tokens)
- **Zapewnienie jakości:** Vitest 2 [10]

1.4. Użyte technologie

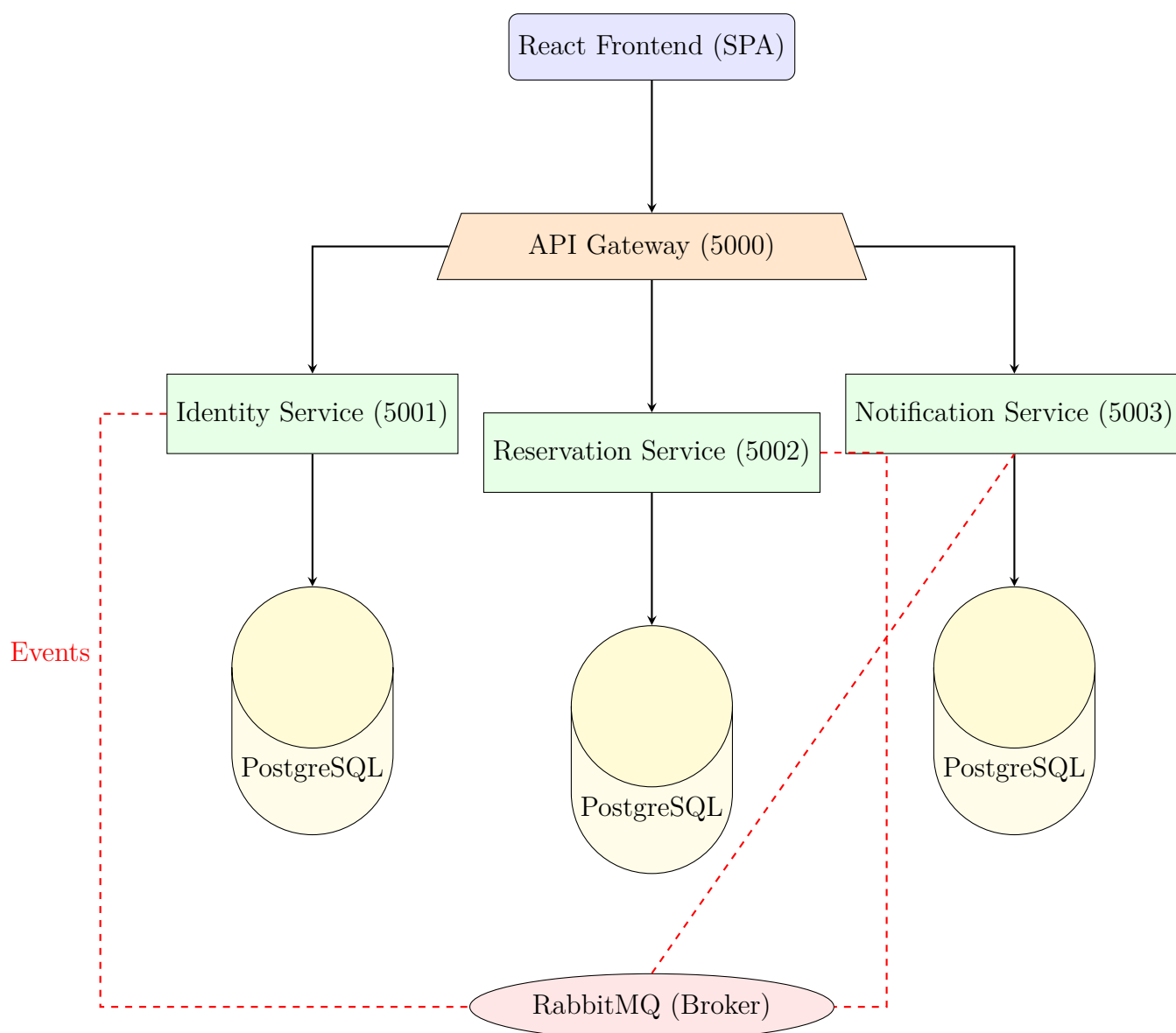
Tabela 1: Stack technologiczny projektu

Kategoria	Technologia	Szczegóły / Uzasadnienie
<i>Backend & Dane</i>		
Platforma	.NET 8.0	Wykorzystanie mikroservisów (Identity oraz Reservation).
ORM	EF Core 8.0	Bezpieczny dostęp do danych i automatyczne migracje bazy.
Baza danych	PostgreSQL 15	Trzy niezależne bazy zapewniające izolację danych i spójność.
Messaging	RabbitMQ 3.12	Broker komunikatów zapewniający asynchroniczną wymianę danych.
<i>Frontend & UI</i>		
Framework	React 19	Najnowsza wersja biblioteki zapewniająca wysoką wydajność UI.
Build Tool	Vite 7	Szybki i nowoczesny system budowania aplikacji SPA.
Komunikacja	Axios 1.x	Klient HTTP z obsługą interceptorów dla tokenów JWT.
Real-time	SignalR 8.0	Komunikacja dwukierunkowa dla powiadomień w czasie rzeczywistym.
<i>Infrastruktura & Bezpieczeństwo</i>		
Autentykacja	JWT	System tokenów Access + Refresh dla bezpiecznego dostępu.
Kontenery	Docker	Konteneryzacja usług przy użyciu Docker Compose.
<i>Zapewnienie Jakości (QA)</i>		
Testy Frontend	Vitest 2	Nowoczesny framework testowy zintegrowany z Vite.
Jakość kodu	ESLint 9	Narzędzie do statycznej analizy i utrzymania standardów kodu.

2 Architektura systemu

2.1. Architektura wysokopoziomowa

Projekt został zrealizowany w modelu mikroserwisowym, co pozwala na pełną izolację odpowiedzialności za poszczególne obszary biznesowe systemu rezerwacji. Całość komunikacji zewnętrznej opiera się na protokole HTTPS i jest zarządzana przez centralny punkt dostępowy, który dba o bezpieczeństwo i odpowiednie przekierowanie ruchu do usług wewnętrznych.



Rysunek 1: Architektura mikroserwisowa systemu rezerwacji

Główne komponenty systemu obejmują:

- **React Frontend (SPA):** Warstwa prezentacji odpowiedzialna za interakcję z

użytkownikiem [2].

- **API Gateway (Port 5000):** Brama sieciowa zarządzająca przepływem żądań do odpowiednich usług [11].
- **Identity Service (Port 5001):** Serwis odpowiedzialny za autentykację i zarządzanie profilami [1].
- **Reservation Service (Port 5002):** Moduł zarządzający firmami, pracownikami oraz procesem planowania wizyt.
- **Notification Service (Port 5003):** Usługa zajmująca się asynchroniczną wysyłką powiadomień.
- **RabbitMQ:** Broker komunikatów umożliwiający asynchroniczną wymianę zdarzeń [6].
- **Bazy Danych (PostgreSQL):** Trzy odizolowane bazy danych zapewniające separację stanów [5].

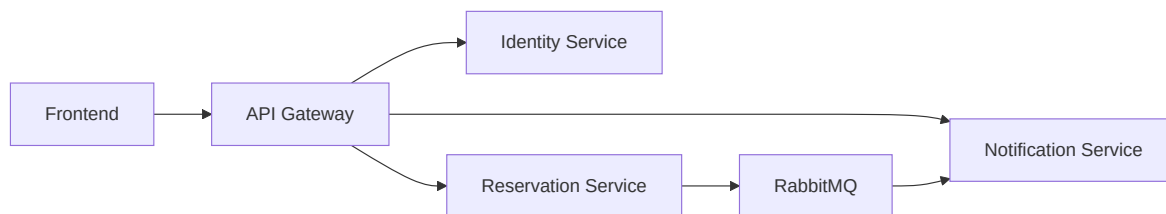
2.2. Wzorce architektoniczne i projektowe

W procesie projektowania systemu wykorzystano wzorce zapewniające wysoką jakość oprogramowania i niezależność komponentów.

2.2.1. API Gateway (Ocelot)

Zastosowano wzorzec bramy API [11], która stanowi jedyny punkt wejścia dla aplikacji frontendowej. Brama odpowiada za:

- **Routing:** Przekierowywanie żądań HTTP do mikroserwisów na podstawie ścieżki adresu.
- **Zarządzanie ruchem (Rate Limiting):** Ograniczenie liczby żądań (100 req/min) w celu ochrony przed przeciążeniem.
- **Obsługa awarii (Circuit Breaker):** Wykorzystanie biblioteki *Polly* do automatycznego odcinania ruchu do serwisów ulegających awarii.
- **JWT Validation:** Centralna weryfikacja poprawności tokenów [9] przed przekazaniem żądania dalej.



Rysunek 2: Architektura komunikacji międzyusługowej (Inter-Service Communication)

2.2.2. CQRS (Command Query Responsibility Segregation)

Wewnątrz mikroservisów rozdzielono operacje modyfikujące dane od operacji odczytu:

- **Commands:** Operacje zmieniające stan bazy danych (np. *CreateAppointment*).
- **Queries:** Operacje pobierające dane, zoptymalizowane pod kątem szybkości wyświetlania (np. *GetAvailableSlots*).

2.2.3. Architektura sterowana zdarzeniami (Event-Driven Architecture)

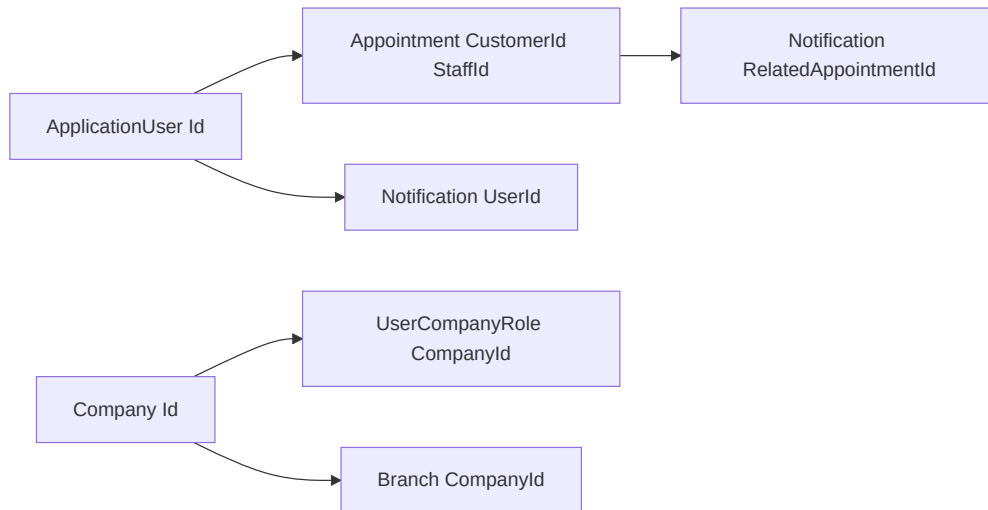
Komunikacja między mikroservisami odbywa się asynchronicznie za pomocą zdarzeń (*Events*) przesyłanych przez RabbitMQ [6]. Przykładowo, *ReservationService* publikuje komunikat po zapisie rezerwacji, który jest konsumowany przez *NotificationService* w celu wysyłki powiadomienia e-mail.

2.2.4. Wzorzec Repozytorium (Repository Pattern)

Zastosowano warstwę abstrakcji między logiką biznesową a bazą danych PostgreSQL. Repozytoria ukrywają szczegóły implementacyjne *Entity Framework Core* [12], co pozwala na łatwą zmianę dostawcy danych bez modyfikacji reguł biznesowych.

2.2.5. Logiczny model powiązań danych

W architekturze mikroservisowej, mimo fizycznej separacji baz danych (PostgreSQL), konieczne jest zachowanie logicznej spójności relacji między obszarami biznesowymi. Na rysunku 3 przedstawiono schemat powiązań, który definiuje sposób identyfikacji obiektów w rozproszonym środowisku.



Rysunek 3: Logiczny model relacji między encjami mikroservisów Identity, Reservation i Notification.

Model ten strukturyzuje przepływ informacji w dwóch kluczowych obszarach:

- **Relacje użytkownika i procesów (Oś górna):** Identyfikator *ApplicationUser Id* z serwisu tożsamości stanowi klucz logiczny dla rezerwacji (*Appointment*), pełniąc rolę klienta (*CustomerId*) lub pracownika (*StaffId*). Zdarzenie utworzenia rezerwacji inicjuje proces w module powiadomień, gdzie encja *Notification* odwołuje się do konkretnej wizyty poprzez *RelatedAppointmentId*.
- **Struktura biznesowa i uprawnienia (Oś dolna):** Centralna encja *Company* definiuje kontekst dla oddziałów (*Branch*) oraz przypisuje użytkownikom konkretne uprawnienia w ramach organizacji (*UserCompanyRole*). Taka konstrukcja zapewnia izolację danych w modelu *multi-tenancy* – użytkownik ma dostęp tylko do tych zasobów, które są powiązane z jego identyfikatorem firmy (*CompanyId*).

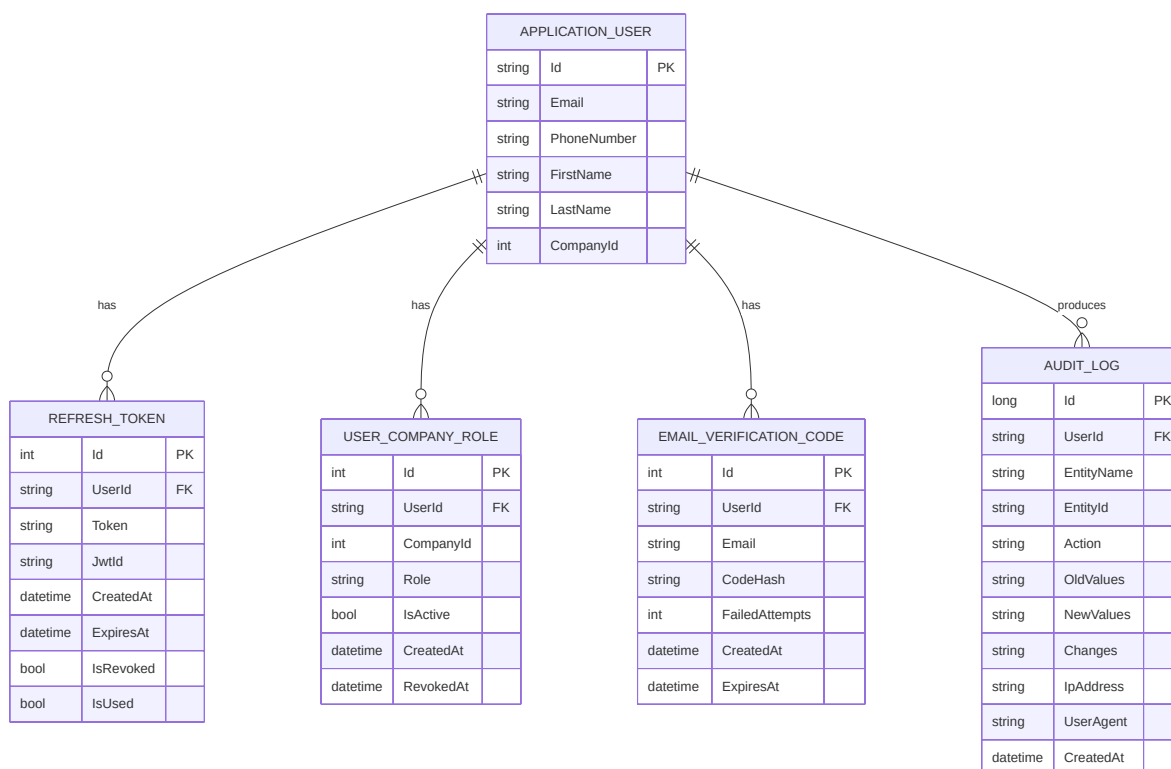
3 Struktura bazy danych

System wykorzystuje relacyjną bazę danych **PostgreSQL** [5] w modelu *Database-per-Service*. Oznacza to, że każdy z trzech głównych mikroservisów posiada własną, odizolowaną bazę danych, co zapobiega powstawaniu silnych powiązań (*tight coupling*) na poziomie schematu danych i umożliwia niezależne skalowanie usług.

3.1. Baza danych tożsamości (IdentityService DB)

Baza ta odpowiada za przechowywanie danych o użytkownikach, ich rolach w systemie oraz bezpieczeństwo sesji [1]. Wykorzystuje ona mechanizmy *Entity Framework Identity*.

- **ApplicationUsers:** Przechowuje dane profilowe, skróty haseł oraz opcjonalne powiązanie z firmą (*CompanyId*).
- **RefreshTokens:** Tabela wspierająca mechanizm bezpiecznego odświeżania sesji. Każdy token jest unikalny i powiązany z konkretnym użytkownikiem [9].
- **AuditLogs:** Rejestruje aktywność użytkowników. Pole *UserId* jest opcjonalne, co pozwala na logowanie akcji systemowych lub nieudanych prób logowania.



Rysunek 4: Diagram związków encji (ERD) bazy danych mikroservisów Identity-Service

3.2. Baza danych rezerwacji (ReservationService DB)

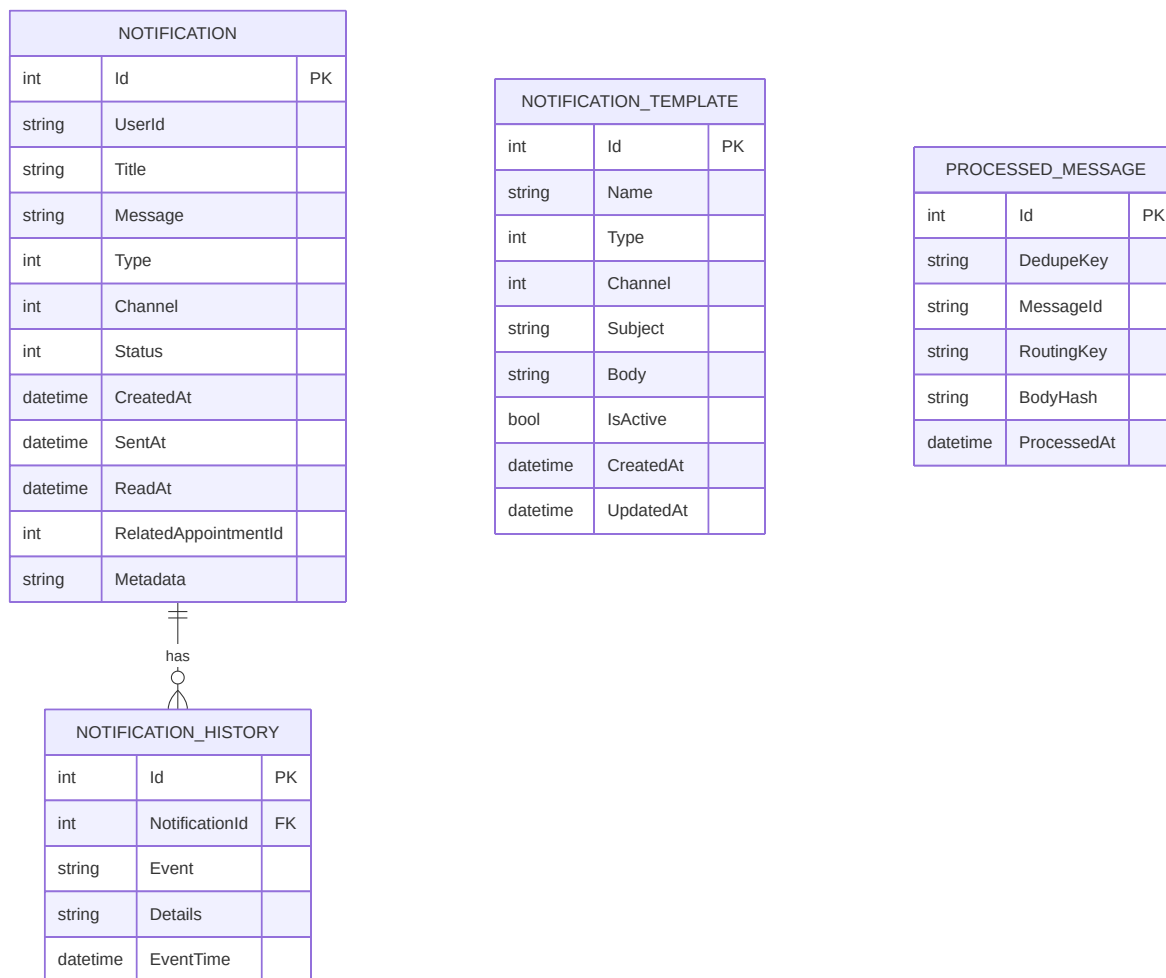
Na rysunku 6 przedstawiono najbardziej rozbudowaną część systemu, zarządzającą logiką biznesową mikro-SaaS. Struktura wspiera model wielonajemności.

- **Branches i Services:** Definiują strukturę firmy. Usługi zawierają parametry takie jak cena oraz czas trwania wraz z czasem buforowym.
- **Schedules i TimeSlots:** Tabele odpowiedzialne za dostępność. *TimeSlot* posiada opcjonalne powiązanie z *AppointmentId* – brak powiązania oznacza, że dany termin jest wolny.
- **Appointments:** Główna tabela rezerwacji. Implementuje mechanizm **Soft Delete**, dzięki czemu rekordy po anulowaniu nie są usuwane fizycznie, co pozwala na zachowanie spójności w tabeli *BranchReviews*.

3.3. Baza danych powiadomień (NotificationService DB)

Baza ta została zoptymalizowana pod kątem wydajnej obsługi komunikatów i zapewnienia ich idempotentności.

- **Notifications:** Przechowuje historię komunikatów *In-App*, e-mail oraz SMS.
- **ProcessedMessages:** Kluczowa tabela dla stabilności systemu. Zawiera pole **DedupeKey**, które zapobiega ponownemu przetworzeniu tego samego zdarzenia z RabbitMQ (idempotentność konsumenta).
- **Templates:** Przechowuje szablony wiadomości z dynamicznymi placeholderami, które są uzupełniane danymi z pola *RelatedAppointmentId*.

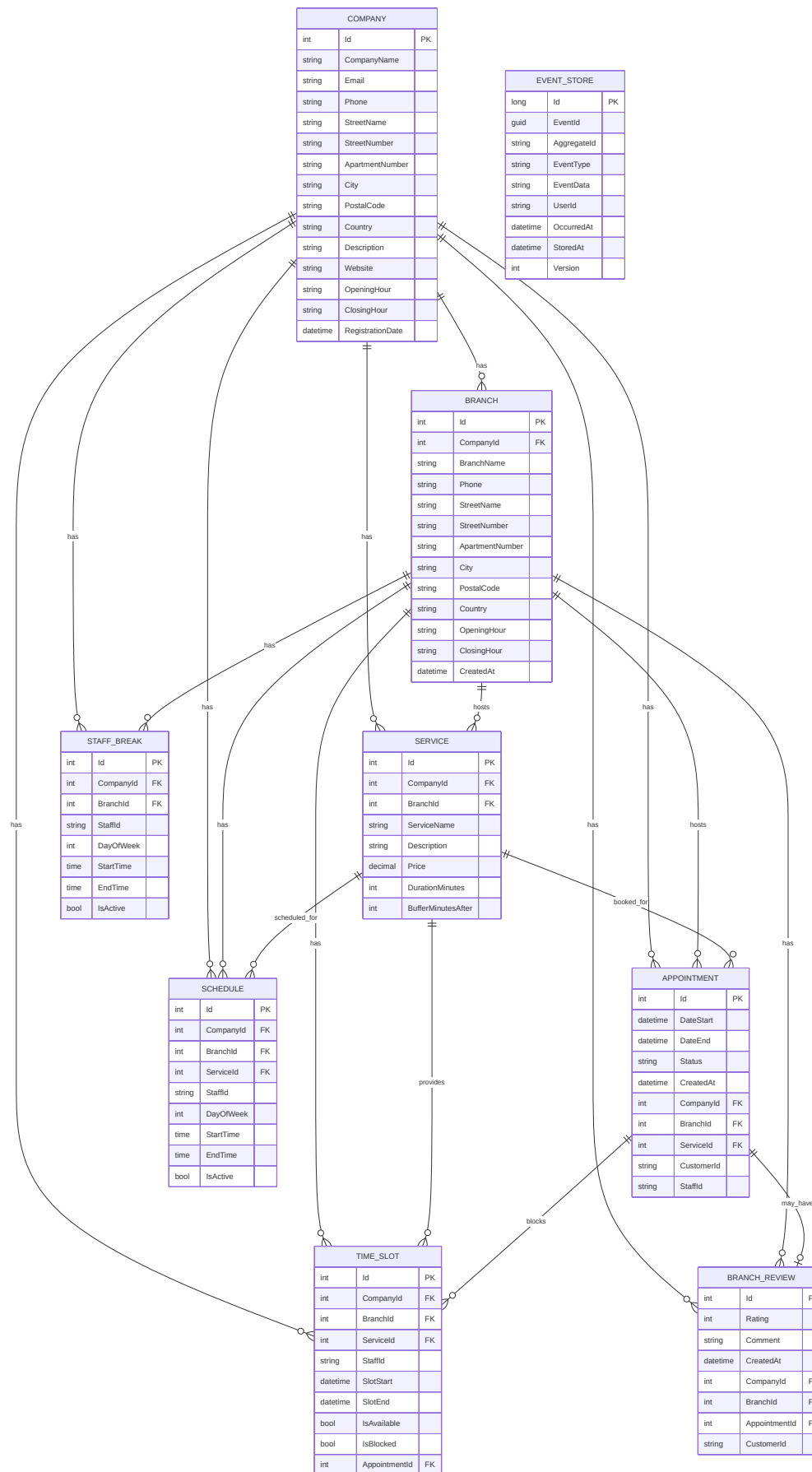


Rysunek 5: Diagram związków encji (ERD) bazy danych mikroservisu Notification-Service

3.4. Integralność danych w architekturze rozproszonej

Z racji fizycznej separacji baz danych, system nie wykorzystuje tradycyjnych więzów integralności (*Foreign Keys*) pomiędzy serwisami. Zamiast tego zastosowano mechanizm **spójności ostatecznej** (*Eventual Consistency*):

1. Identyfikatory użytkowników (GUID) są przekazywane w tokenach JWT [9] i zapisywane jako zwykłe pola typu UUID w bazie rezerwacji.
2. Zmiany danych (np. zmiana statusu rezerwacji) są propagowane asynchronicznie przez broker komunikatów RabbitMQ [6].
3. Mechanizm *Audit Log* w każdej bazie pozwala na odtworzenie stanu systemu w przypadku niespójności.



Rysunek 6: Diagram związków encji (ERD) bazy danych mikroservisu Reservation-Service

4 Implementacja kluczowych funkcjonalności

4.1. Bezpieczeństwo i zarządzanie tożsamością (JWT)

Fundamentem bezpieczeństwa systemu jest bezstanowa autoryzacja oparta na standardzie *JSON Web Token* [9], zaimplementowana w mikroserwisie *IdentityService*. System wykorzystuje mechanizm dwóch tokenów w celu zbalansowania bezpieczeństwa i wygody użytkownika:

- **Access Token:** Posiada krótki czas życia wynoszący **15 minut**, co minimalizuje ryzyko w przypadku przejęcia tokenu.
- **Refresh Token:** Przechowywany bezpiecznie w bazie danych, z czasem wygaśnięcia ustalonym na **7 dni**, umożliwiając automatyczne odświeżanie sesji bez konieczności ponownego logowania.

Obsługa cyklu życia sesji odbywa się poprzez dedykowany kontroler *RefreshTokenController*. Implementuje on kluczowe operacje bezpieczeństwa, takie jak odświeżanie tokenów (*Refresh*) oraz ich unieważnianie (*Revoke*), co pozwala na natychmiastowe zakończenie sesji użytkownika w przypadku wylogowania lub wykrycia naruszenia bezpieczeństwa.

```
1 [HttpPost("refresh")]
2 public async Task<IActionResult> RefreshToken([FromBody] RefreshTokenRequest
   request)
3 {
4     if (string.IsNullOrEmpty(request.RefreshToken))
5     {
6         return BadRequest("Refresh token is required");
7     }
8
9     // Delegacja logiki odswiezania do serwisu JWT
10    var result = await _jwtService.RefreshTokenAsync(request.RefreshToken);
11
12    if (result == null)
13    {
14        _logger.LogWarning("Invalid refresh token attempt");
15        return Unauthorized("Invalid refresh token");
16    }
17
18    return Ok(result);
19 }
20
```

```
21 [HttpPost("revoke")]
22 [Authorize]
23 public async Task<IActionResult> RevokeToken([FromBody] RevokeTokenRequest
    request)
24 {
25     // Pobranie ID zalogowanego uzytkownika z tokenu
26     var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
27
28     // Weryfikacja wlasciciela tokenu przed jego uniewaznieniem
29     var tokenOwnerId = await _context.RefreshTokens
30         .AsNoTracking()
31         .Where(rt => rt.Token == request.RefreshToken)
32         .Select(rt => rt.UserId)
33         .FirstOrDefaultAsync();
34
35     if (!string.Equals(tokenOwnerId, userId, StringComparison.Ordinal) && !
        User.IsInRole("Admin"))
36     {
37         return Forbid();
38     }
39
40     await _jwtService.RevokeTokenAsync(request.RefreshToken);
41     return Ok(new { message = "Token revoked successfully" });
42 }
```

Listing 1: Implementacja punktów końcowych do zarządzania sesją (RefreshTokenController.cs)

4.2. Zarządzanie strukturą biznesową i ofertą usług

System wspiera model wielooddziałowy, korzystając z *Entity Framework Core* [12]. W celu zapewnienia integralności danych historycznych, w module tym zaimplementowano mechanizm **Soft Delete**. Zamiast trwałego usuwania rekordów z bazy danych (np. przy usuwaniu usługi lub oddziału), system ustawia flagę *IsDeleted*. Dzięki temu stare rezerwacje zachowują powiązania do usług, które nie są już oferowane nowym klientom.

4.3. Algorytm wyliczania dostępności i rezerwacji terminów

Moduł rezerwacji wykorzystuje zaawansowany mechanizm filtrowania dostępności w czasie rzeczywistym, zaimplementowany w ramach mikroserwisu. Logika ta opiera się na dwuetapowej weryfikacji: najpierw pobierane są teoretyczne okna czasowe z serwisu harmonogramów, a następnie system przeprowadza rygorystyczną kontrolę kolizji z istniejącymi już rezerwacjami.

```
1 // Pobranie dostępnych slotów dla danego dnia z ScheduleService
2 var daySlots = await _scheduleService.GetAvailableSlotsAsync(
3     service.CompanyId,
4     service.BranchId,
5     service.Id,
6     dto.DateStart);
7
8 // Sprawdzenie, czy wybrana godzina startu znajduje się w wygenerowanej liście
9 var isAllowed = daySlots.Any(s => s.StaffId == dto.StaffId && s.Start == dto.
    DateStart);
10 if (!isAllowed)
11 {
12     return BadRequest("Selected start time is not available");
13 }
14
15 // Obliczenie czasu zakończenia wraz z wymaganą przerwą po usłudze (buforem)
16 var computedEnd = dto.DateStart.AddMinutes(service.DurationMinutes);
17 var newBlockedEnd = computedEnd.AddMinutes(service.BufferMinutesAfter);
18
19 // Pobranie istniejących rezerwacji pracownika (z pominięciem anulowanych)
20 var existingAppointments = await _context.Appointments
21     .AsNoTracking()
22     .Include(a => a.Service)
23     .Where(a => a.CompanyId == service.CompanyId
24         && a.BranchId == service.BranchId
25         && a.StaffId == dto.StaffId
26         && a.Status != "cancelled")
27     .ToListAsync();
28
29 // Algorytm wykrywania kolizji przedziałów czasowych (Overlap Detection)
30 var conflictingAppointment = existingAppointments.Any(a =>
31 {
```

```
32     var buffer = a.Service?.BufferMinutesAfter ?? 0;
33     var aBlockedEnd = a.DateEnd.AddMinutes(buffer);
34
35     // Warunek nachodzenia na siebie dwóch przedziałów czasowych
36     return dto.DateStart < aBlockedEnd && newBlockedEnd > a.DateStart;
37 });
38
39 if (conflictingAppointment)
40 {
41     return BadRequest("This time slot is already booked");
42 }
```

Listing 2: Implementacja logiki wykrywania kolizji i walidacji slotów (AppointmentsController.cs)

4.4. Asynchroniczny obieg zdarzeń i powiadomień

Komunikacja w systemie odbywa się dwutorowo, co zapewnia wysoką responsywność oraz niezawodność dostarczania informacji. Głównym kanałem wymiany danych między mikroservisami jest asynchroniczna szyna danych oparta na brokerze **RabbitMQ**. Drugim filarem jest technologia **SignalR Hub**, wykorzystywana do komunikacji w czasie rzeczywistym z interfejsem użytkownika.

Kluczowym elementem mikroservisów *NotificationService* jest mechanizm dynamicznego generowania treści powiadomień w oparciu o szablony. Jak przedstawiono w kontrolerze *NotificationsController*, system wspiera wstrzykiwanie zmiennych (*placeholders*), co pozwala na automatyczną personalizację wiadomości (np. imię klienta, data wizyty) przed ich wysłaniem kanałem e-mail lub SMS.

```
1 // Metoda odpowiedzialna za tworzenie powiadomienia z wykorzystaniem szablonu
2 [HttpPost]
3 public async Task<ActionResult<Notification>> CreateNotification(
4     CreateNotificationDto dto)
5 {
6     // Pobranie zdefiniowanego szablonu (np. przypomnienie o wizycie)
7     NotificationTemplate? template = null;
8     if (dto.TemplateId.HasValue)
9     {
10         template = await _context.NotificationTemplates.FindAsync(dto.
11             TemplateId.Value);
12     }
```

```
12     var notification = new Notification
13     {
14         UserId = dto.UserId,
15         Title = dto.Title ?? template?.Subject ?? "Powiadomienie",
16         Message = dto.Message ?? template?.Body ?? "",
17         Status = NotificationStatus.Pending,
18         RelatedAppointmentId = dto.RelatedAppointmentId
19     };
20
21     // Proces wstrzykiwania dynamicznych danych (placeholders) do tre'sci
22     if (dto.Placeholders != null)
23     {
24         foreach (var placeholder in dto.Placeholders)
25         {
26             notification.Title = notification.Title.Replace($"{{{placeholder.
27             Key}}}", placeholder.Value);
28             notification.Message = notification.Message.Replace($"{{{
29             placeholder.Key}}}", placeholder.Value);
30         }
31     }
32
33     _context.Notifications.Add(notification);
34     await _context.SaveChangesAsync();
35
36     // Przekazanie do asynchronicznego nadajnika (np. integracja z SMTP/SMS
37     Gateway)
38     await _notificationSender.SendNotificationAsync(notification);
39
40     return CreatedAtAction(nameof(GetNotification), new { id = notification.Id
41     }, notification);
42 }
```

Listing 3: Mechanizm personalizacji powiadomień i obsługi szablonów (NotificationsController.cs)

4.5. Dziennik audytowy i śledzenie zmian (*Audit Log*)

Wszystkie operacje wykonywane przez użytkowników o podwyższonych uprawnieniach (Admin, Owner, Manager) są monitorowane przez system audytowy. Rejestrowane są nie tylko same akcje, ale również metadane, takie jak identyfikator sesji i adres IP. Dane te

są kluczowe dla zachowania transparentności w modelu *multi-tenancy*, gdzie wiele firm współdzieli zasoby systemowe.

Dostęp do danych audytowych jest ściśle limitowany poprzez mechanizmy autoryzacji oparte na politykach (*Policies*). Jak przedstawiono w kontrolerze *CompanyAuditController*, system rygorystycznie weryfikuje przynależność użytkownika do danej firmy, uniemożliwiając wgląd w historię operacji innych organizacji.

```
1 [HttpGet("company/{companyId}")]
2 [Authorize(Policy = "CompanyOwnerOrAdmin")]
3 public async Task<IActionResult> GetCompanyAudit(int companyId, [FromQuery]
4     int take = 200)
5 {
6     // Weryfikacja uprawnień w modelu multi-tenancy
7     if (!User.IsInRole("Admin"))
8     {
9         var claimCompanyId = User.FindFirst("CompanyId")?.Value;
10        if (!string.Equals(claimCompanyId, companyId.ToString(),
11            StringComparison.Ordinal))
12        {
13            // Proba dostępu do danych innej firmy zostaje zablokowana
14            return Forbid();
15        }
16    }
17
18    // Pobranie danych przez dedykowany serwis audytowy
19    var logs = await _audit.GetCompanyAuditAsync(companyId, take);
20    return Ok(logs);
21 }
```

Listing 4: Implementacja bezpiecznego dostępu do logów audytowych (*CompanyAuditController.cs*)

5 Warstwa prezentacji – Implementacja funkcjonalności Frontend (React)

Warstwa kliencka systemu została zbudowana przy użyciu biblioteki **React 19** [2]. Do budowy i optymalizacji projektu wykorzystano narzędzie Vite [3]. Poniżej opisano technologiczną realizację kluczowych modułów oraz funkcjonalności interfejsu użytkownika, skupiając się na logice biznesowej i integracji z mikroservisami za pomocą biblioteki Axios [4].

5.1. Moduł Użytkownika – Proces rejestracji i rezerwacji

Interfejs klienta został zaprojektowany z myślą o prostocie obsługi. Proces rejestracji obejmuje weryfikację adresu e-mail za pomocą cyfrowego kodu OTP, co zapewnia bezpieczeństwo tożsamości. Najważniejszym elementem jest formularz rezerwacji, który dynamicznie oblicza dostępność slotów czasowych na podstawie danych z serwisu rezerwacji.

```
1  const validateContact = (next) => {
2    const email = String(next.customerEmail || "").trim();
3    const phone = sanitizePhoneNumberInput(next.customerPhone);
4
5    const emailOk = !email || /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
6    const phoneOk = !phone || /^[+\d{8,15}$/.test(phone);
7
8    return {
9      customerEmail: emailOk ? "" : "booking.invalidEmail",
10     customerPhone: phoneOk ? "" : "booking.invalidPhone"
11   };
12 };
13
14 const filteredAvailableSlots = useMemo(() => {
15   if (!form.date || !form.staffId) return [];
16   return availableSlots.filter((s) =>
17     String(s.staffId || "") === String(form.staffId)
18   );
19 }, [availableSlots, form.date, form.staffId]);
```

Listing 5: Walidacja danych kontaktowych oraz filtrowanie slotów (BookingPage.js)

5.2. Zarządzanie profilem i Dashboard klienta

Użytkownik po zalogowaniu ma dostęp do pulpitu (*Dashboard*), który agreguje informacje o nadchodzących wizytach. System umożliwi również zarządzanie danymi profilowymi, wykorzystując zaawansowane funkcje normalizacji danych wejściowych, co gwarantuje spójność bazy danych. Testowanie logiki komponentów frontendowych zrealizowano przy pomocy środowiska Vitest [10].

```
1  const normalizePostalCodeInput = (value) => {  
2    const digits = String(value || '').replace(/\D/g, '').slice(0, 5);  
3    if (digits.length <= 2) return digits;  
4    return `${digits.slice(0, 2)}-${digits.slice(2)}`;  
5  };  
6  
7  const formatPhoneDisplay = (value) => {  
8    const raw = String(value || '').trim();  
9    const digits = raw.replace(/\D/g, '').slice(0, 12);  
10   if (!digits) return raw.startsWith('+') ? '+' : '';  
11  
12   const country = digits.slice(0, 3);  
13   const rest = digits.slice(3);  
14   const groups = rest.match(/.{1,3}/g) || [];  
15   return `+${country} ${groups.join('-')}`;  
16  };
```

Listing 6: Normalizacja kodu pocztowego i formatowanie numeru telefonu (AccountPage.js)

5.3. Moduł Firmowy – Zarządzanie uprawnieniami i kadrą

Panel administracyjny firmy dynamicznie dostosowuje zakres dostępnych funkcjonalności na podstawie roli użytkownika (RBAC – *Role-Based Access Control*). Logika frontendu rozróżnia uprawnienia właściciela (*Owner*), managera oraz pracownika, korzystając z informacji zaszyfrowanych w standardzie JSON Web Token [9].

```
1  const companyRole = tokenManager.getUser()?.companyRole;  
2  const userRoles = tokenManager.getUser()?.roles || [];  
3  const isAdmin = userRoles.includes("Admin");  
4  const isOwner = companyRole === "Owner";  
5  const isManager = companyRole === "Manager";  
6  
7  // Definicja dostępu do akcji krytycznych  
8  const canManageEmployees = isAdmin || isOwner || isManager;
```

```
9  const canEditCompany = isAdmin || isOwner;
10
11  const handleUpdateUserRole = async (userId, newRole) => {
12    try {
13      await companyUsersAPI.updateUserRole(userId, newRole);
14      await loadEmployees();
15    } catch (err) {
16      console.error("Failed to update role", err);
17    }
18  };
```

Listing 7: Logika weryfikacji uprawnień wewnątrz organizacji (CompanyPanel.js)

5.4. Zarządzanie harmonogramami i przerwami pracowników

Ważną funkcjonalnością jest wizualizacja czasu pracy. Użytkownik może definiować dni i godziny pracy oraz konfigurować przerwy (*Staff Breaks*). Do obsługi powiadomień o zmianach w czasie rzeczywistym przewidziano integrację z technologią SignalR [7].

```
1  const toggleDay = (day) => {
2    const d = Number(day);
3    const current = normalizeDays(normalizedValue);
4    const next = current.includes(d)
5      ? current.filter((x) => x !== d)
6      : [...current, d];
7    onChange(normalizeDays(next));
8  };
9
10  const handleCreateStaffBreak = async (e) => {
11    e.preventDefault();
12    const daysToCreate = normalizeDays(staffBreakFilters.dayOfWeek);
13    await Promise.all(
14      daysToCreate.map((d) => staffBreaksAPI.create({ ...payload, dayOfWeek: d
15        }))
16    );
17  };
```

Listing 8: Obsługa komponentu wyboru dni tygodnia (CompanyPanel.js)

5.5. Moduł Administracyjny i szablony powiadomień

Administratorzy systemowi zarządzają szablonami wiadomości, które wspierają dynamiczne wstrzykiwanie zmiennych. Pozwala to na personalizację powiadomień e-mail i SMS wysyłanych przez mikroserwis powiadomień zintegrowany z systemem kolejek RabbitMQ [6].

```
1 const insertTemplateToken = (token) => {
2   const field = activeTemplateField === "subject" ? "subject" : "body";
3   const el = field === "subject" ? subjectRef.current : bodyRef.current;
4   const currentValue = String(editingTemplate?.[field] ?? "");
5
6   if (el && typeof el.selectionStart === "number") {
7     const start = el.selectionStart;
8     const end = el.selectionEnd;
9     const next = `${currentValue.slice(0, start)}${token}${currentValue.slice(
10      end)}`;
11     handleTemplateFormChange(field, next);
12   }
13 };
```

Listing 9: Mechanizm wstawiania tokenów do edytora szablonów (AdminPanel.js)

5.6. Integracja z geokodowaniem i mapami

W widoku szczegółów usługi zaimplementowano interaktywną mapę **OpenStreetMap**. System przelicza adres tekstowy na współrzędne geograficzne i wyświetla lokalizację salonu, co znacząco poprawia użyteczność aplikacji. Całość procesu wdrażania frontendu jako kontenera została opisana w dokumentacji Docker [8].

```
1 useEffect(() => {
2   const geocode = async () => {
3     try {
4       const res = await geocodeAPI.geocode(osmQuery);
5       if (res.data?.found) {
6         setMapCoords({ lat: res.data.lat, lon: res.data.lon });
7       }
8     } catch (err) {
9       setMapCoords(null);
10    }
11  };
12  if (osmQuery) geocode();
13 }, [osmQuery]);
```

Listing 10: Geokodowanie adresu i generowanie widoku mapy (ServiceDetailsPage.js)

6 Konteneryzacja i środowisko uruchomieniowe

W celu zapewnienia powtarzalności środowiska oraz uproszczenia procesu wdrażania, system został w pełni skonteneryzowany przy użyciu technologii **Docker** [8]. Pozwala to na izolację poszczególnych mikroservisów wraz z ich zależnościami w lekkich kontenerach, co eliminuje problem różnic między środowiskami programistycznymi a produkcyjnymi. Obrazy wynikowe są przechowywane i zarządzane w ramach rejestru *GitHub Container Registry* [13].

6.1. Konfiguracja wielokontenerowa (Docker Compose)

Do zarządzania całym ekosystemem usług wykorzystano narzędzie **Docker Compose**. Pozwala ono na zdefiniowanie wszystkich komponentów systemu (mikroservisy, bazy danych, brokerzy komunikatów) w jednym pliku konfiguracyjnym `docker-compose.yml` oraz ich jednoczesne uruchomienie przy pomocy jednej komendy.

```
1 # Budowa obraz'ow i uruchomienie kontener'ow w tle
2 docker-compose up --build -d
```

Listing 11: Uruchomienie całego stosu technologicznego systemu

6.2. Charakterystyka zdefiniowanych usług

Zgodnie z architekturą systemu, w pliku konfiguracyjnym zdefiniowano następujące grupy usług:

- **Infrastruktura danych (PostgreSQL):** Wykorzystano obraz *postgres:15-alpine* [5]. Kontener został skonfigurowany tak, aby przy starcie automatycznie tworzył trzy odrębne bazy danych (*identitydb*, *reservationdb*, *notificationdb*) za pomocą skryptu inicjalizacyjnego. Trwałość danych zapewniono poprzez wolumeny (*Volumes*).
- **Broker komunikatów (RabbitMQ):** Zastosowano obraz z panelem zarządzania (*management-alpine*) [6], co pozwala na monitorowanie kolejek zdarzeń w czasie rzeczywistym na porcie 15672.
- **Warstwa mikroservisów (.NET):** Serwisy *Identity*, *Reservation* oraz *Notification* są budowane z lokalnych plików *Dockerfile* w oparciu o SDK .NET 8 [1].

- **API Gateway (Ocelot):** Wykorzystano bibliotekę Ocelot [11], która pełni rolę centralnego punktu wejścia, mapując zewnętrzny port 5000 na wewnętrzną infrastrukturę kontenerów.
- **Narzędzia pomocnicze (MailHog):** Kontener służący do przechwytywania i testowania wiadomości e-mail w środowisku deweloperskim, co pozwala na bezpieczną weryfikację systemu powiadomień bez wysyłki realnych wiadomości.

[Image of Docker Compose architecture for microservices showing PostgreSQL, RabbitMQ, and .NET containers connected via a bridge network]

6.3. Zarządzanie zależnościami i Healthchecks

W konfiguracji zastosowano mechanizm **Healthcheck**, który monitoruje stan gotowości usług krytycznych. Mikroserwisy posiadają instrukcję `depends_on` z warunkiem `service_healthy`, co gwarantuje, że aplikacja .NET spróbuje nawiązać połączenie z bazą dopiero wtedy, gdy silnik bazy danych przejdzie pomyślnie test gotowości.

```
1 reservation_api:
2   build:
3     context: ./ReservationService
4   depends_on:
5     postgres_db:
6       condition: service_healthy
7     rabbitmq:
8       condition: service_healthy
```

Listing 12: Fragment konfiguracji Docker Compose obsługujący zależności

6.4. Izolacja sieciowa

Wszystkie kontenery działają w ramach dedykowanej sieci wirtualnej typu *bridge*. Pozwala to na komunikację między serwisami przy użyciu nazw ich usług (np. `Host=postgres_db`) zamiast zmiennych adresów IP. Takie rozwiązanie zapewnia bezpieczną izolację — bazy danych nie są wystawione na świat zewnętrzny, a dostęp do nich mają wyłącznie autoryzowane mikroserwisy wewnątrz sieci Docker.

6.5. Automatyzacja zarządzania systemem (Skrypty Manage)

W celu maksymalnego uproszczenia pracy z systemem, przygotowano dedykowane skrypty zarządcze: `manage.bat` (dla systemu Windows) oraz `manage.sh` (dla systemów Unix). Stanowią one warstwę abstrakcji nad surowymi komendami silnika Docker oraz narzędzia Vite [3].

```
1 # Uruchomienie backendu w kontenerach oraz frontendu React
2 manage.bat all
```

Listing 13: Uruchomienie kompletnego środowiska za pomocą skryptu zarządczego

Wykorzystanie powyższego skryptu wykonuje sekwencyjnie następujące zadania:

- **Weryfikacja środowiska:** Sprawdzenie dostępności niezbędnych narzędzi (Docker, Node.js).
- **Orkiestracja kontenerów:** Wywołanie `docker-compose` w celu zbudowania i uruchomienia infrastruktury.
- **Uruchomienie warstwy prezentacji:** Automatyczny start serwera deweloperskiego Vite [3] dla aplikacji frontendowej.

Dzięki tej automatyzacji proces wdrożenia i testowania (w tym skanowanie podatności obrazów narzędziem Trivy [14]) został zintegrowany z procesem ciągłej integracji w GitHub Actions [15].

7 Testowanie i proces CI/CD

Kluczowym elementem wytwarzania nowoczesnego oprogramowania jest zapewnienie jego jakości poprzez automatyzację procesów weryfikacyjnych. W niniejszym systemie zaimplementowano wielopoziomowe podejście do testowania oraz potok ciągłej integracji (ang. *Continuous Integration*).

7.1. Testy automatyczne

System został objęty testami automatycznymi zarówno w warstwie serwerowej, jak i klienckiej, co pozwala na szybkie wykrywanie błędów regresji.

7.1.1. Testy warstwy backendowej (xUnit)

Dla mikroservisów Identity, Reservation oraz Notification przygotowano dedykowane projekty testowe wykorzystujące framework **xUnit** [16]. Testy te skupiają się na weryfikacji logiki biznesowej, poprawności mapowania obiektów oraz działaniu punktów końcowych API.

- **Walidacja domeny:** Sprawdzanie reguł biznesowych, np. zakazu rezerwacji terminów w przeszłości.
- **Testy integracyjne:** Weryfikacja poprawnej komunikacji z bazą danych PostgreSQL przy użyciu biblioteki Entity Framework Core [12].

7.1.2. Testy warstwy frontendowej (Vitest)

W części klienckiej zastosowano nowoczesny framework testowy **Vitest** [10]. Testy obejmują kluczowe komponenty interfejsu użytkownika oraz mechanizmy nawigacji.

- **Weryfikacja komponentów:** Testowanie renderowania elementów interfejsu takich jak ikony i przyciski nawigacyjne.
- **Logika autoryzacji:** Testowanie ochrony ścieżek (*ProtectedRoute*), zapewniające, że nieautoryzowani użytkownicy nie mają dostępu do paneli administracyjnych.

7.2. Potok CI/CD w środowisku GitHub Actions

Proces integracji i wdrażania został w pełni zautomatyzowany przy użyciu usługi **GitHub Actions** [15]. Skonfigurowany potok (*pipeline*) uruchamia się automatycznie przy każdym zdarzeniu *push* lub *pull request*.

Proces CI obejmuje następujące etapy:

1. **Przywracanie i budowa (Restore & Build):** Automatyczne pobieranie zależności bibliotecznych oraz kompilacja kodu źródłowego mikroservisów i aplikacji frontendowej.
2. **Uruchomienie testów:** Wykonanie wszystkich testów jednostkowych i integracyjnych. Niepowodzenie któregośkolwiek z nich skutkuje przerwaniem procesu wdrażania.
3. **Skanowanie bezpieczeństwa (Security Scan):** Wykorzystanie narzędzia **Trivy** [14] do analizy obrazów i kodu pod kątem znanych podatności (CVE).
4. **Budowa obrazów (Build Images):** Po pomyślnym przejściu testów, system buduje obrazy kontenerowe i publikuje je w rejestrze **GHCR** [13].

7.3. Weryfikacja integracyjna (Smoke Tests)

Poza testami jednostkowymi, wdrożono skrypty typu **Smoke Tests**. Ich zadaniem jest uruchomienie całego stosu technologicznego i sprawdzenie podstawowej drożności systemu, w tym:

- Poprawności działania routingu w **API Gateway**.
- Sprawdzenia dostępności (*health check*) wszystkich punktów końcowych API mikroservisów.
- Symulacji kluczowych kroków procesu rezerwacji i autentykacji.

8 Podsumowanie i perspektywy rozwoju

Niniejsza praca pozwoliła na zaprojektowanie i zaimplementowanie skalowalnego systemu rezerwacji usług w modelu Mikro-SaaS. Wykorzystanie architektury mikroserwisowej opartej na platformie .NET 8 [1] oraz bibliotece React 19 [2] umożliwiło stworzenie rozwiązania charakteryzującego się wysoką wydajnością oraz czytelnym podziałem odpowiedzialności między poszczególnymi modułami.

8.1. Wnioski z realizacji projektu

Proces wytwórczy oraz przeprowadzone testy automatyczne [10] pozwoliły na sformułowanie następujących wniosków inżynierskich:

- **Efektywność komunikacji asynchronicznej:** Zastosowanie brokera komunikatów RabbitMQ [6] pozwoliło na całkowite odseparowanie serwisu rezerwacji od modułu powiadomień. Dzięki temu proces rezerwacji terminu nie jest blokowany przez czasochłonne operacje wysyłki wiadomości e-mail.
- **Izolacja danych:** Model *Database-per-Service* z wykorzystaniem bazy PostgreSQL [5] zagwarantował, że błędy w obrębie jednego schematu danych (np. logów powiadomień) nie wpływają na stabilność krytycznych tabel rezerwacji.
- **Wydajność interfejsu:** Wykorzystanie narzędzia Vite [3] oraz mechanizmów normalizacji danych wejściowych (np. telefonów i kodów pocztowych) znacząco poprawiło komfort pracy użytkownika (*User Experience*) oraz jakość danych trafiających do systemu.
- **Bezpieczeństwo:** Implementacja mechanizmu *Refresh Tokens* oraz centralna walidacja JWT na poziomie API Gateway [11] zapewniły standard bezpieczeństwa zgodny z zaleceniami RFC 7519 [9].

8.2. Kierunki dalszego rozwoju systemu

Choć system w obecnej fazie jest w pełni funkcjonalny, architektura pozwala na jego dalszą rozbudowę w następujących obszarach:

- **Integracja z kalendarzami zewnętrznymi:** Implementacja dwukierunkowej synchronizacji z Google Calendar oraz Outlook za pomocą dedykowanych API.
- **System płatności online:** Integracja z platformą Stripe [4] w celu umożliwienia pobierania przedpłat za rezerwacje oraz zarządzania subskrypcjami firm korzystających z modelu SaaS.

- **Aplikacja mobilna:** Stworzenie natywnego interfejsu dla pracowników w technologii React Native, co ułatwiłoby zarządzanie harmonogramem w terenie.
- **Rozbudowa modułu powiadomień:** Dodanie obsługi wiadomości SMS oraz powiadomień typu *Push* z wykorzystaniem technologii SignalR [7].

8.3. Podsumowanie końcowe

Projekt udowodnił, że zastosowanie wzorców takich jak CQRS, API Gateway oraz komunikacja sterowana zdarzeniami pozwala na budowę systemów gotowych do obsługi dużego ruchu i wielu niezależnych organizacji jednocześnie. Praca nad systemem pozwoliła na zdobycie praktycznego doświadczenia w zakresie orkiestracji kontenerów Docker [8] oraz automatyzacji procesów CI/CD [15], co stanowi solidną podstawę dla nowoczesnej inżynierii oprogramowania.

Bibliografia

- [1] Microsoft Learn, “.net 8 documentation,” <https://learn.microsoft.com/pl-pl/dotnet/>, 2025, dostęp: 15.10.2025.
- [2] Meta Platforms, “React 19 documentation,” <https://react.dev>, 2025, dostęp: 15.10.2025.
- [3] Vite, “Vite 7 guide,” <https://vite.dev/guide/>, 2025, dostęp: 10.12.2025.
- [4] Axios Project, “Axios documentation,” <https://axios-http.com/docs/intro>, 2025, dostęp: 12.12.2025.
- [5] The PostgreSQL Global Development Group, “Postgresql 15 documentation,” <https://www.postgresql.org/docs/15/index.html>, 2024, dostęp: 10.12.2025.
- [6] Broadcom (VMware), “Rabbitmq 3.12 documentation,” <https://www.rabbitmq.com/docs>, 2025, dostęp: 10.12.2025.
- [7] Microsoft Learn, “Asp.net core signalr 8.0 introduction,” <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction>, 2025, dostęp: 15.12.2025.
- [8] Docker Inc., “Docker documentation,” <https://docs.docker.com/>, 2024, dostęp: 18.11.2025.
- [9] M. B. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt) - rfc 7519,” <https://datatracker.ietf.org/doc/html/rfc7519>, 2015, dostęp: 08.01.2026.
- [10] Vitest, “Vitest 2 guide,” <https://vitest.dev/guide/>, 2025, dostęp: 20.12.2025.
- [11] Ocelot Team, “Ocelot api gateway documentation,” <https://ocelot.readthedocs.io/>, 2024, dostęp: 08.01.2026.
- [12] Microsoft Learn, “Testing applications that use entity framework core,” <https://learn.microsoft.com/en-us/ef/core/testing/>, 2025, dostęp: 04.01.2026.
- [13] GitHub Inc., “Working with the container registry,” <https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry>, 2025, dostęp: 05.01.2026.
- [14] Aqua Security, “Trivy vulnerability scanner documentation,” <https://aquasecurity.github.io/trivy/>, 2025, dostęp: 03.01.2026.
- [15] GitHub Inc., “Github actions documentation,” <https://docs.github.com/en/actions>, 2025, dostęp: 02.01.2026.

-
- [16] xUnit.net Project, “xunit.net documentation,” <https://xunit.net/docs/getting-started/netcore/cmdline>, 2025, dostęp: 05.01.2026.